

Prédiction de la Qualité de l’Air dans les Villes Marocaines

Approche Machine Learning et simulation d’apprentissage fédéré (FedAvg)

Réalisé par :

Yessef Mouad

Khallabi Bilal

Module : Apprentissage Fédéré

Outils : Python, scikit-learn, pandas, OpenWeatherMap API

Rapport de projet

Table des matières

1	Introduction	2
1.1	Contexte	2
1.2	Objectifs du projet	2
1.3	Villes étudiées	2
1.4	Architecture générale du pipeline	2
2	Étape 1 – Collecte des données	4
2.1	Objectif	4
2.2	Sources de données	4
2.3	Démarche technique	4
2.4	Structure du fichier de sortie	4
2.5	Extrait de code	5
3	Étape 2 – Préparation des données	6
3.1	Objectif	6
3.2	Étapes de nettoyage	6
3.3	Variable cible	6
3.4	Fichiers générés	7
3.5	Extrait de code	7
4	Étape 3 – Construction et évaluation des modèles	8
4.1	Objectif	8
4.2	Modèles utilisés	8
4.3	Protocole expérimental	8
4.4	Métriques d'évaluation	8
4.5	Résultats	9
4.6	Extrait de code	9
5	Étape 4 (bonus) – Simulation d'apprentissage fédéré	10
5.1	Motivation	10
5.2	Principe de l'algorithme FedAvg	10
5.3	Adaptation pratique avec scikit-learn	10
5.4	Partitionnement des données	11
5.5	Agrégation FedAvg	11
5.6	Résultats et comparaison	11
6	Discussion et limites	13
6.1	Limites identifiées	13
6.2	Pistes d'amélioration	13
7	Conclusion	14

Chapitre 1

Introduction

1.1 Contexte

La pollution atmosphérique constitue un enjeu environnemental et sanitaire majeur dans les grandes villes marocaines. Le suivi de la qualité de l’air, notamment à travers l’indicateur PM2.5 (particules fines de diamètre inférieur à 2,5 micromètres), permet d’anticiper les pics de pollution et d’informer les politiques publiques ainsi que les citoyens.

Ce projet a pour objectif de construire un pipeline complet de *data science*, depuis la collecte de données réelles via une API publique jusqu’à la construction de modèles prédictifs de *machine learning*, en passant par le nettoyage et la préparation des données. Une extension optionnelle explore les bases de l’**apprentissage fédéré** (*federated learning*), une approche permettant d’entraîner un modèle global sans centraliser les données provenant de différentes villes.

1.2 Objectifs du projet

- Collecter des données météorologiques et de qualité de l’air réelles pour au moins trois villes marocaines, via l’API OpenWeatherMap.
- Nettoyer, structurer et préparer ces données pour l’apprentissage automatique.
- Entraîner et comparer plusieurs modèles de régression pour prédire la concentration de PM2.5.
- Simuler un scénario d’apprentissage fédéré où chaque ville agit comme un client indépendant, et comparer les performances avec l’approche centralisée.

1.3 Villes étudiées

Quatre villes marocaines ont été retenues pour cette étude :

Ville	Latitude	Longitude
Casablanca	33.5731	-7.5898
Rabat	34.0209	-6.8416
Marrakech	31.6295	-7.9811
Fès	34.0331	-5.0003

1.4 Architecture générale du pipeline

Le projet est organisé en quatre étapes successives, chacune correspondant à un script Python indépendant et réutilisable :

1. `01_collect_data.py` – Collecte automatisée des données via l'API OpenWeatherMap.
2. `02_prepare_data.py` – Nettoyage, encodage et normalisation des données.
3. `03_build_model.py` – Entraînement et évaluation de plusieurs modèles de régression.
4. `04_federated_learning.py` – Simulation d'un apprentissage fédéré (FedAvg) entre les villes.

Cette architecture modulaire garantit la reproductibilité du pipeline : chaque script lit le fichier produit par l'étape précédente et génère lui-même un fichier de sortie consommé par l'étape suivante.

Chapitre 2

Étape 1 – Collecte des données

2.1 Objectif

Cette étape consiste à interroger en continu l'API publique **OpenWeatherMap** afin de récupérer, pour chacune des quatre villes, des données météorologiques et des données de qualité de l'air, puis à les stocker dans un fichier CSV structuré.

2.2 Sources de données

Deux points de terminaison (*endpoints*) de l'API OpenWeatherMap sont utilisés :

- **Weather API** (`/data/2.5/weather`) : température, ressenti, humidité, pression, vitesse et direction du vent, couverture nuageuse, description météo.
- **Air Pollution API** (`/data/2.5/air_pollution`) : indice de qualité de l'air (AQI) et concentrations des polluants (CO, NO, NO₂, O₃, SO₂, PM2.5, PM10, NH₃).

2.3 Démarche technique

- La clé API est chargée depuis une variable d'environnement (`OPENWEATHER_API_KEY`) afin d'éviter de l'exposer directement dans le code.
- Un mécanisme de **nouvelle tentative** (*retry*) avec un maximum de 3 essais et un délai de 10 secondes entre chaque tentative gère les erreurs réseau ou les indisponibilités temporaires de l'API.
- Chaque ligne collectée combine les données météo et qualité de l'air, horodatées (`timestamp`) et arrondies à l'heure (`timestamp_hour`) afin de faciliter la fusion ultérieure des données.
- Les doublons (même ville, même heure) sont éliminés à chaque sauvegarde grâce à `drop_duplicates(subs`
- Une boucle infinie (`while True`) relance la collecte toutes les heures (`COLLECTION_INTERVAL_SECONDS = 3600`), permettant de constituer un historique de données sur plusieurs jours.

2.4 Structure du fichier de sortie

Le fichier `morocco_air_quality_data.csv` contient, pour chaque observation, les colonnes suivantes : `timestamp`, `timestamp_hour`, `city`, `lat`, `lon`, `temperature`, `feels_like`, `humidity`, `pressure`, `wind_speed`, `wind_deg`, `clouds`, `weather_main`, `weather_description`, `aqi`, `co`, `no`, `no2`, `o3`, `so2`, `pm2_5`, `pm10`, `nh3`.

2.5 Extrait de code

La fonction ci-dessous illustre la construction d'une ligne de données pour une ville donnée, en combinant les deux appels API :

```
1 def collect_one_city(city_name, lat, lon):
2     weather = get_weather_data(lat, lon)
3     time.sleep(2)
4     air = get_air_quality_data(lat, lon)
5
6     air_item = air["list"][0]
7     components = air_item["components"]
8
9     row = {
10         "timestamp": now.strftime("%Y-%m-%d %H:%M:%S"),
11         "city": city_name,
12         "temperature": weather["main"].get("temp"),
13         "humidity": weather["main"].get("humidity"),
14         "pm2_5": components.get("pm2_5"),
15         "pm10": components.get("pm10"),
16         # ... autres variables meteo / pollution
17     }
18     return row
```

Listing 2.1 – Extrait – construction d'une observation (01_collect_data.py)

Le code complet de cette étape correspond au script 01_collect_data.py.

Chapitre 3

Étape 2 – Préparation des données

3.1 Objectif

Cette étape transforme les données brutes collectées en un jeu de données propre, cohérent et exploitable par les algorithmes de *machine learning*.

3.2 Étapes de nettoyage

1. **Suppression des doublons** : doublons exacts, puis doublons logiques (même ville + même heure).
2. **Conversion des types** : les colonnes `timestamp` sont converties au format date/heure ; les colonnes numériques (température, humidité, polluants, etc.) sont forcées au type numérique, les valeurs invalides devenant `NaN`.
3. **Traitement des valeurs manquantes** :
 - Les lignes sans valeur pour la cible (`pm2_5`) ou les variables jugées essentielles (température, humidité, pression) sont supprimées.
 - Les autres valeurs numériques manquantes sont imputées par la **médiane** de leur colonne, un choix robuste aux valeurs extrêmes.
 - Les variables catégorielles manquantes sont remplacées par la valeur "Unknown".
4. **Création de variables temporelles** : extraction de l'année, du mois, du jour et de l'heure à partir du `timestamp`, permettant au modèle de capter une éventuelle saisonnalité ou un effet horaire sur la pollution.
5. **Encodage des variables catégorielles** (`city`, `weather_main`, `weather_description`) via un encodage *one-hot* (`pd.get_dummies`).
6. **Normalisation** des variables numériques à l'aide d'un `StandardScaler` (centrage-réduction), particulièrement utile pour les modèles linéaires.

3.3 Variable cible

La variable cible retenue pour ce projet est **PM2.5** (`pm2_5`), traitée comme un problème de **régression**, conformément à l'une des deux options proposées dans l'énoncé du projet (PM2.5 en régression ou classe AQI en classification).

3.4 Fichiers générés

- `morocco_air_quality_data_clean.csv` : version nettoyée mais non encodée, conservée pour analyse exploratoire et pour l'étape d'apprentissage fédéré (qui doit conserver la colonne `city` en clair afin de partitionner les clients).
- `morocco_air_quality_data_model_ready.csv` : version encodée et normalisée, prête à être utilisée directement par les modèles de régression.

3.5 Extrait de code

```
1 # Imputation par la mediane pour les variables numeriques
2 for col in numeric_columns:
3     if col in df.columns and df[col].isna().sum() > 0:
4         df[col] = df[col].fillna(df[col].median())
5
6 # Encodage one-hot des variables categorielles
7 df_model = pd.get_dummies(
8     df,
9     columns=[col for col in categorical_columns if col in df.columns],
10    drop_first=True
11 )
12
13 # Normalisation (centrage-reduction)
14 scaler = StandardScaler()
15 X_scaled[numeric_features_in_X] = scaler.fit_transform(X[numeric_features_in_X])
```

Listing 3.1 – Extrait – imputation et encodage (`02_prepare_data.py`)

Le code complet de cette étape correspond au script `02_prepare_data.py`.

Chapitre 4

Étape 3 – Construction et évaluation des modèles

4.1 Objectif

L'objectif de cette étape est d'entraîner plusieurs modèles de régression supervisée afin de prédire la concentration de PM2.5, puis de comparer leurs performances.

4.2 Modèles utilisés

Trois modèles, de complexité croissante, ont été comparés :

- **Régression linéaire** (`LinearRegression`) : modèle de référence (*baseline*), simple et interprétable.
- **Arbre de décision** (`DecisionTreeRegressor`) : modèle non linéaire capable de capter des interactions entre variables.
- **Forêt aléatoire** (`RandomForestRegressor`, 100 arbres) : ensemble d'arbres de décision, généralement plus robuste et plus performant que ses composants pris individuellement.

4.3 Protocole expérimental

- Le jeu de données prêt (`morocco_air_quality_data_model_ready.csv`) est divisé en un ensemble d'entraînement (80 %) et un ensemble de test (20 %), avec une graine aléatoire fixe (`random_state=42`) pour assurer la reproductibilité.
- Chaque modèle est entraîné sur les mêmes données d'entraînement, puis évalué sur le même ensemble de test.

4.4 Métriques d'évaluation

Trois métriques standards pour un problème de régression sont calculées :

- **MAE** (*Mean Absolute Error*) : erreur absolue moyenne entre les valeurs prédites et réelles.
- **RMSE** (*Root Mean Squared Error*) : racine carrée de l'erreur quadratique moyenne, plus sensible aux grandes erreurs que la MAE.
- **R²** (coefficient de détermination) : proportion de la variance de la cible expliquée par le modèle (1 = prédiction parfaite).

4.5 Résultats

Les résultats numériques obtenus sont sauvegardés automatiquement dans le fichier `step3_model_results.csv`. Le tableau ci-dessous reprend la structure de ce fichier ; les valeurs doivent être complétées avec les résultats obtenus lors de l'exécution du script sur votre jeu de données final.

Modèle	MAE	RMSE	R ²
Régression linéaire	<i>à compléter</i>	<i>à compléter</i>	<i>à compléter</i>
Arbre de décision	<i>à compléter</i>	<i>à compléter</i>	<i>à compléter</i>
Forêt aléatoire	<i>à compléter</i>	<i>à compléter</i>	<i>à compléter</i>

TABLE 4.1 – Comparaison des performances des modèles (issu de `step3_model_results.csv`)

De manière générale, on attend que la Forêt aléatoire surpasse la Régression linéaire et l'Arbre de décision seul, car elle réduit la variance par agrégation (*bagging*) tout en captant les relations non linéaires entre les conditions météorologiques et la concentration de PM2.5.

4.6 Extrait de code

```

1 models = {
2     "Linear Regression": LinearRegression(),
3     "Decision Tree Regressor": DecisionTreeRegressor(random_state=42),
4     "Random Forest Regressor": RandomForestRegressor(n_estimators=100,
5         random_state=42)
6 }
7 for model_name, model in models.items():
8     model.fit(X_train, y_train)
9     y_pred = model.predict(X_test)
10
11     mae = mean_absolute_error(y_test, y_pred)
12     rmse = np.sqrt(mean_squared_error(y_test, y_pred))
13     r2 = r2_score(y_test, y_pred)

```

Listing 4.1 – Extrait – boucle d'entraînement et d'évaluation (`03_build_model.py`)

Le code complet de cette étape correspond au script `03_build_model.py`.

Chapitre 5

Étape 4 (bonus) – Simulation d'apprentissage fédéré

5.1 Motivation

Dans un contexte réel, les données de qualité de l'air pourraient être détenues localement par chaque ville (ou par un capteur local) sans pouvoir être centralisées, pour des raisons de confidentialité, de coût de transfert ou de gouvernance des données. L'**apprentissage fédéré** permet d'entraîner un modèle global performant *sans jamais faire circuler les données brutes*, seuls les paramètres des modèles locaux étant partagés.

5.2 Principe de l'algorithme FedAvg

L'algorithme **Federated Averaging (FedAvg)** se déroule en plusieurs rounds. À chaque round :

1. Le modèle global est conceptuellement envoyé à chaque client (ici, chaque ville).
2. Chaque client entraîne un modèle local sur ses propres données.
3. Les paramètres locaux (coefficients et intercept, pour une régression linéaire) sont renvoyés et agrégés par une **moyenne pondérée** par le nombre d'échantillons de chaque client :

$$\theta_{\text{global}} = \sum_{k=1}^K \frac{n_k}{N} \theta_k \quad (5.1)$$

où θ_k représente les paramètres du modèle local du client k , n_k son nombre d'échantillons, N le nombre total d'échantillons sur l'ensemble des clients, et K le nombre de clients (ici, $K = 4$ villes).

5.3 Adaptation pratique avec scikit-learn

La bibliothèque **scikit-learn** ne permet pas nativement d'initialiser un modèle **LinearRegression** avec des coefficients prédéfinis avant l'entraînement, ni de réaliser un entraînement incrémental round par round. Le script contourne cette limite de façon pédagogique :

- à chaque round, un modèle linéaire est ré-entraîné *localement et indépendamment* pour chaque ville sur l'intégralité de ses données ;
- les coefficients et intercepts obtenus sont ensuite agrégés par FedAvg pour produire le modèle global du round ;

- les prédictions du modèle global agrégé sont calculées manuellement via le produit matriciel $\hat{y} = X \cdot \text{coef} + \text{intercept}$, puisque ce modèle n'est plus un objet `LinearRegression` entraîné directement par `scikit-learn`.

Cette approche reste une **simulation simplifiée** : pour une régression linéaire entraînée par moindres carrés, chaque round reproduit en réalité le même résultat local (le modèle converge directement vers la solution optimale dès le premier entraînement). L'intérêt pédagogique du script réside donc surtout dans la mécanique de partitionnement par client et d'agrégation FedAvg, plus que dans une amélioration itérative réelle du modèle global – ce qui serait le cas avec un modèle entraîné par descente de gradient (ex. réseau de neurones), où chaque round d'agrégation ferait effectivement progresser le modèle global.

5.4 Partitionnement des données

Le partitionnement des clients fédérés est réalisé après le découpage train/test, en conservant la colonne `city` d'origine :

```

1 X_train, X_test, y_train, y_test, city_train, city_test = train_test_split(
2     X, y, city_series,
3     test_size=0.2, random_state=42, stratify=city_series
4 )
5
6 clients = sorted(city_train.unique())
7 client_data = {}
8 for city in clients:
9     idx = city_train[city_train == city].index
10    client_data[city] = {
11        "X": X_train.loc[idx],
12        "y": y_train.loc[idx],
13        "n_samples": len(idx)
14    }
```

Listing 5.1 – Extrait – partitionnement des clients (04_federated_learning.py)

La stratification (`stratify=city_series`) garantit que chaque ville est représentée proportionnellement dans les ensembles d'entraînement et de test.

5.5 Agrégation FedAvg

```

1 total_samples = sum(local_sizes)
2 weighted_coef_sum = np.zeros_like(global_coef, dtype=float)
3 weighted_intercept_sum = 0.0
4
5 for coef, intercept, n_samples in zip(local_coefs, local_intercepts, local_sizes):
6     weight = n_samples / total_samples
7     weighted_coef_sum += weight * coef
8     weighted_intercept_sum += weight * intercept
9
10 global_coef = weighted_coef_sum
11 global_intercept = weighted_intercept_sum
```

Listing 5.2 – Extrait – agrégation pondérée FedAvg (04_federated_learning.py)

5.6 Résultats et comparaison

Le script produit automatiquement un fichier `federated_vs_centralized_results.csv` comparant les deux approches. La structure des résultats est la suivante :

Approche	MAE	RMSE	R ²
Régression linéaire centralisée	<i>à compléter</i>	<i>à compléter</i>	<i>à compléter</i>
Régression linéaire fédérée (FedAvg)	<i>à compléter</i>	<i>à compléter</i>	<i>à compléter</i>

TABLE 5.1 – Comparaison centralisé vs fédéré

On s'attend à ce que le modèle fédéré obtienne des performances **proches** du modèle centralisé, ce qui illustrerait l'intérêt principal du *federated learning* : préserver la confidentialité des données locales sans sacrifier significativement la qualité du modèle global.

Le code complet de cette étape correspond au script `04_federated_learning.py`.

Chapitre 6

Discussion et limites

6.1 Limites identifiées

- **Volume de données** : la qualité des modèles dépend fortement de la durée de collecte ; quelques jours de données horaires restent limités pour capter pleinement la variabilité saisonnière de la pollution.
- **Simulation simplifiée du federated learning** : comme expliqué au chapitre 4, l'utilisation d'une régression linéaire entraînée par moindres carrés ne permet pas d'illustrer la convergence progressive typique de FedAvg sur plusieurs rounds.
- **Dépendance à une seule source de données** : l'API OpenWeatherMap fournit des estimations de qualité de l'air qui peuvent différer des mesures de stations officielles marocaines.

6.2 Pistes d'amélioration

- Étendre la période de collecte sur plusieurs semaines, voire plusieurs mois.
- Ajouter davantage de villes pour renforcer le scénario fédéré.
- Utiliser un modèle entraînable de manière incrémentale (ex. `SGDRegressor` ou un petit réseau de neurones) pour un FedAvg plus représentatif.
- Explorer la classification des niveaux d'AQI en complément de la régression sur PM2.5.

Chapitre 7

Conclusion

Ce projet a permis de mettre en œuvre un pipeline complet de *data science*, de la collecte de données réelles via une API publique jusqu'à la comparaison de plusieurs modèles de régression, en passant par une étape rigoureuse de nettoyage et de préparation des données. L'extension optionnelle sur l'apprentissage fédéré a permis d'illustrer concrètement le principe de l'algorithme FedAvg et de comparer une approche centralisée à une approche distribuée par ville.

Ce travail constitue une base solide, reproductible et documentée, pouvant être étendue à davantage de villes, de polluants ou de modèles plus sophistiqués.